Lecture 2-2: Differential Equation Sampling

/afs/ictp/public/l/lhuet/NM1_Lecture4

Leon Huet
lhuet@ictp.it
adapted from Zejian Li slides

12. Nov. 2025

Introduction

A **Differential equation** is an equation where unknown functions are related to their derivatives.

- **Ordinary Differential Equations (ODE)**: Differential equations for functions depending on only one variable.
  - Order of ODE: the highest appearing order of derivative of the function
  - System of ODE: coupled differential equation for multiple functions (each depending on only one variable).

$$\text{Bacteria growth} \quad \frac{\mathrm{d}w}{\mathrm{d}t} = \eta w \quad \text{is a 1st order ODE}$$

$$\text{(Damped) harmonic oscillator} \quad \frac{\mathrm{d}^2x}{\mathrm{d}t^2} = -\gamma\frac{\mathrm{d}x}{\mathrm{d}t} - \frac{k}{m}x \quad \text{is a 2nd order ODE}$$

- **Partial Differential Equations (PDE)**: Differential equations for functions depending on multiple variables. For example: Maxwell differential equations are a system of first order PDE.

Numerical Methods for Differential Equation

Often analytical solutions are complicated, hard to find, or unknown. Even more often it is known that there exist no analytical solutions

In that case we have to accept that the numerical approximations we can make, come with come with some errors. We do what we call sampling :

- Set with the initial conditions.
- Take a small step: Calculate an approximate value of the function for a small increment of the independent variable.
- Take another small step: Calculate the next approximate value of the function for another small increment of the independent variable.
- . . . and so on . . .

The main inconvenient of doing that is that you reiterate the approximation a each step and apply is on data that are already approximated. $\rightarrow$ The errors are cumulated

Euler Method - Idea

For 1st order ODE, which have the form:

$$\frac{\mathrm{d}x}{\mathrm{d}t} = f(t, x) \quad \text{with} \quad x(t_0) = x_0$$

For example:

$$\text{radioactive decay:} \quad \frac{\mathrm{d}x}{\mathrm{d}t} = f(\cancel{t}, x) = -\gamma x$$

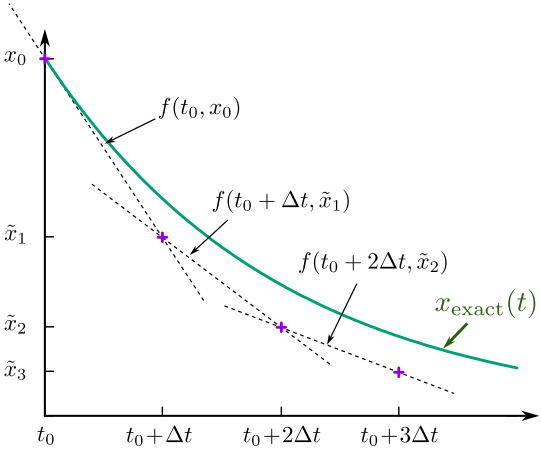
Consider the Taylor expansion at $t = t_0$ of the solution $x(t)$:

$$x(t_0 + \Delta t) = x(t_0) + \Delta t \cdot \left.\frac{\mathrm{d}x}{\mathrm{d}t}\right|_{t=t_0} + \mathcal{O}(\Delta t^2)$$

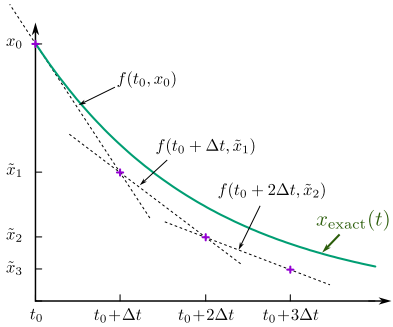
$$x(t_0 + \Delta t) \quad \approx \quad x(t_0) + f(t_0, x_0) \cdot \Delta t$$

Euler Method - Sketch

$$x(t_0 + \Delta t) \quad \approx \quad x(t_0) + f(t_0, x_0) \cdot \Delta t$$



Euler Method - Algorithm



Algorithm: Euler Method

Input: function $f(t, x)$, initial values t_0, x_0, step size Δt, and number of steps N

1. set $x := x_0$ and $t := t_0$
2. repeat N times:
   - set $fx := f(t, x)$
   - set $x := x + fx \cdot \Delta t$
   - set $t := t + \Delta t$

Output: the sequence of t and x approximating the solution of $\frac{\mathrm{d}x}{\mathrm{d}t} = f(t, x)$.

Euler Method - Fortran Implementation Sketch

```fortran
subroutine euler_method(t0, x0, dt, N, tlist, xlist)
  ! ... variable declarations and allocations ...
  x = x0
  t = t0
  tlist(1) = t
  xlist(1) = x
  do i = 1, N
    fx = f(t,x)
    x = x + fx * dt
    t = t + dt
    tlist(i+1) = t
    xlist(i+1) = x
  end do
end subroutine euler_method
```

Higher Order ODE $\rightarrow$ System of 1st Order ODE

Euler method and midpoint method only work for 1st order ODEs. However, ...

It is always possible to rewrite a higher order ODE as a system of 1st order ODEs!

For example, the equation of motion for the angle θ of a friction-less (stiff) pendulum

$$\frac{\mathrm{d}^2\theta}{\mathrm{d}t^2} = -\sin(\theta)$$

can be rewritten as the system of two first order ODEs (for the angle θ and the angular velocity $\dot{\theta}$ (or v_θ as you want):

$$\frac{\mathrm{d}\theta}{\mathrm{d}t} = \dot{\theta}$$
$$\frac{\mathrm{d}\dot{\theta}}{\mathrm{d}t} = -sin(\theta)$$

Euler Method - Algorithm for a System

Algorithm: Euler Method for a System x and y

Input: functions $f_x(t, x, y)$, $f_y(t, x, y)$, initial values t_0, x_0, y_0, step size Δt, and number of steps N

1. set $x := x_0$, $y := y_0$ and $t := t_0$
2. repeat N times:
   - set $fx := f_x(t, x, y)$
   - set $fy := f_y(t, x, y)$
   - set $x_{new} := x + fx(t, x, y) \cdot \Delta t$
   - set $y_{new} := y + fy(t, x, y) \cdot \Delta t$
   - set $x := x_{new}; y := y_{new}$
   - set $t := t + \Delta t$

Output: the sequence of t, x, and y which approximates the solution of

$$\frac{\mathrm{d}x}{\mathrm{d}t} = f_x(t, x, y) \qquad \frac{\mathrm{d}y}{\mathrm{d}t} = f_y(t, x, y).$$

Improvement from Euler: The Verlet method

Generic form of the equation:

$$\frac{\mathrm{d}^2\boldsymbol{x}(t)}{\mathrm{d}t^2} = \boldsymbol{a}(\boldsymbol{x}(t))$$

Idea of the algorithm is to split evaluation in 3 parts:

- Evolve velocity for half a step ($\frac{\Delta t}{2}$):

$$\boldsymbol{v}\left(t + \frac{1}{2}\Delta t\right) \simeq \boldsymbol{v}(t) + \frac{1}{2}\boldsymbol{a}(\boldsymbol{x}(t))\Delta t$$

- Evolve position for a full step using the mid-point velocity:

$$\boldsymbol{x}(t + \Delta t) \simeq \boldsymbol{x}(t) + \boldsymbol{v}\left(t + \frac{1}{2}\Delta\right)\Delta t$$

- Evolve velocity for another half step using acceleration at updated position:

$$\boldsymbol{v}(t + \Delta t) \simeq \boldsymbol{v}\left(t + \frac{1}{2}\Delta t\right) + \frac{1}{2}\boldsymbol{a}\boldsymbol{x}(t + \Delta t)\Delta t$$

Verlet Method: summary

Algorithm: velocity-Verlet Method

Input: acceleration function $\boldsymbol{a}(\boldsymbol{x})$, initial values t_0, $\boldsymbol{x}_0$, $\boldsymbol{v}_0$ step size Δt, and final time t_f.

1. set $t = t_0$, $\boldsymbol{x} = \boldsymbol{x}_0$, $\boldsymbol{v} = \boldsymbol{v}_0$.
2. repeat $N = \lceil t_f / t_0 \rceil$ times:
   - set $\boldsymbol{v} = \boldsymbol{v} + \boldsymbol{a}(\boldsymbol{x})\Delta t/2$,
   - set $\boldsymbol{x} = \boldsymbol{x} + \boldsymbol{v}\Delta t$,
   - set $\boldsymbol{v} = \boldsymbol{v} + \boldsymbol{a}(\boldsymbol{x})\Delta t/2$.

Output: the sequence of t, $\boldsymbol{x}$ and $\boldsymbol{v}$ approximating the solution of $\frac{\mathrm{d}^2\boldsymbol{x}}{\mathrm{d}t^2} = \boldsymbol{a}(\boldsymbol{x})$.

Comparison with Euler method

Input: Same as left

1. Same as left
2. repeat N times:
   - set $\boldsymbol{a}_t = \boldsymbol{a}(\boldsymbol{x})$,
   - set $\boldsymbol{x} = \boldsymbol{x} + \boldsymbol{v}\Delta t$,
   - set $\boldsymbol{v} = \boldsymbol{v} + \boldsymbol{a}_t\Delta t$.

Output: the sequence of t, $\boldsymbol{x}$ and $\boldsymbol{v}$.

- Good numerical stability and other important physical properties: time reversibility and preservation of the symplectic form on phase space, at no significant additional computational cost over the simple Euler method.

Assignment 13

Write a `Fortran` program that solves the equation of motion for a frictionless physical pendulum using the Euler and velocity-Verlet method: $\frac{\mathrm{d}^2 x}{\mathrm{d}t^2} = -\sin(x)$

- Solve with the initial conditions `t0=0, x0=1.0, v0=0.0` and `dt = 0.1, tf=30`.
- The program should create two files (`eurler.txt` and `verlet.txt`) with the result in three columns: `t`, `x(t)` and `v(t)`.
- Plot the results x versus t with your favorite plotting program and compare the Euler and Verlet outputs.

Hints:

- Keep it simple first: begin with euler only
- Use the slides 6 to 9 as supports to code the `euler_method` subroutine.
- Create a copy of your `euler_method` subroutine and evolve it into a `verlet_method` subroutine.

Submit the graphs and your code as `Ass13.YourLastName.f90` to `lhuet@ictp.it` before the next lesson.

Assignment 13: Bonus

- Expand your subroutines and solve the equation of motion for a point mass in a gravitational field (moving in a 2D plane): $\frac{\mathrm{d}^2\boldsymbol{x}}{\mathrm{d}t^2} = -\boldsymbol{x}/|\boldsymbol{x}|^3$ using the euler and verlet methods, with initial conditions `t0=0, x0=(/2.0, 0.0/), v0 = (/0.0, 0.5/)` and `dt = 0.1, tf=30`. Plot the trajectory [$x(1)$ vs $x(2)$] and compare the performance between the two methods.

Reminder for plotting:

- You can use whatever program you like to plot the dynamics. A very simple way is to use `gnuplot`.
  - If the file 'data.txt' has two columns t and x, then you can plot $x(t)$ with:
    `$ gnuplot -p -e "plot 'data.txt' using 1:2 with lines"`
  - If the file 'data.txt' has three columns t, x, and v, then you can plot $x(t)$ and $v(t)$ with:
    `$ gnuplot -p -e "plot for [col=2:3] 'data.txt' using 1:col with lines"`

Side note:

- Usually we need a much smaller `dt` to obtain reliable results. Here we use the relatively large `dt=0.1` to better compare the performance of the difference methods.